

What did the company want to understand?

The company was not entirely sure what should be analyzed. The goal of the project was to provide a general view of the company's financial health based on the data they were able to share with me.

Why does this matter for their business?

This is important because the company needs a clear understanding of its current situation and what can be improved to support growth—not only financially, but also operationally through hiring, taking on more contracts, and purchasing additional equipment.

What limitations existed?

The first major issue I encountered was not only that the available data was limited to purchases and received payments, but also that the information was spread across 82 columns and thousands of rows in Excel worksheets (.xlsx). To use the data effectively in Power BI, it first needed to be converted into a structured format—preferably CSV—so it could be cleaned, transformed, and analyzed more flexibly.

Project Background

Objective:

The client did not have a predefined analytical scope. Instead, the engagement was open-ended: review the available financial data and produce a general assessment of the company's financial health, using whatever raw material could be handed over.

Business Relevance:

Despite the lack of a specific ask, this analysis was highly valuable to the business. A clear view of financial performance gives leadership the insight needed to make informed decisions around growth — whether that means hiring new staff, pursuing additional contracts, or investing in equipment. In short, understanding the numbers is a prerequisite for scaling the business, both financially and operationally.

Constraints & Challenges:

The primary limitation was the state of the source data:

- The only data available consisted of purchasing records and payments received.
- This information was spread across **82 columns and roughly 3,000 rows** in an unstructured Excel (XLSX) worksheet.
- Before any analysis could begin, the data needed to be converted into a clean, structured **CSV format** suitable for ingestion into Power BI — enabling reliable restructuring, transformation, and flexible modelling downstream.

Data & Methodology:

- What data I had? (dates, payments, receipts)
- What data was not available? Useful for measuring project performance and profitability: Project ID, Project name, Project type, Location, Start date, Planned completion date, Actual completion date, Project status, Planned budget, Actual cost, Final profit or loss of project, Change orders, Retention amount, Completion percentage.

Questions this can answer:

- Which projects are profitable?
- Which types of projects generate the most revenue?
- Are projects completed on time?
- Which projects exceed their budgets?

Revenue and payments:

- Project ID
- Payment status
- Payment method
- Outstanding balance
- Possible analysis:
- Accounts receivable
- Late payments
- Average payment delay
- Revenue by project

Possible analysis:

- Late payments
- Average payment delay
- Revenue by client or project
- Cash-flow trends

Expenses and purchases:

- Purchase ID
- Project ID
- Material category
- Description
- Quantity
- Unit price
- Delivery cost
- Payment status

Possible analysis:

- Material costs by project
- Spending by supplier(in comparison)
- Cost trends over time
- Budget versus actual spending
- Most expensive materials
- Unexpected cost increases

Labor and employees

- Project ID
- Hours worked
- Overtime hours
- Hourly rate
- Labor cost

- Start and end dates

(There are usually many more questions in case of many employees like contractor or permanent employee status, absence or leave records, Employee ID etc)

Possible analysis:

- Labor cost by project
- Overtime trends
- Cost per project hour
- Productivity by project
- Whether labor costs are exceeding the budget

Materials and inventory:

- Material ID
- Material name
- Category
- Supplier
- Quantity purchased
- Quantity used
- Quantity remaining
- Unit of measurement
- Unit cost
- Reorder level
- Delivery date
- Waste or damaged quantity

Possible analysis:

- Material usage
- Waste percentage
- Stock levels

- Materials most frequently purchased
- Supplier price comparisons

Equipment and vehicles: important for understanding operational costs.

- Equipment ID
- Equipment type and all related information
- Maintenance cost
- Usage hours
- Project assigned
- Breakdown dates
- Downtime

Possible analysis:

- Equipment cost by project
- Owned versus rented equipment
- Maintenance trends
- Equipment utilization
- Downtime
- Cost of equipment breakdowns

Sales and quotations: useful for understanding business growth.

- Quote ID
- Project type
- Estimated value
- Estimated cost
- Quoted margin
- Lead source

Possible analysis:

- Quote win rate
- Most profitable project types
- Average quote value
- Difference between estimated and actual project profitability

Schedule and progress data:

- Task ID
- Project ID
- Task name
- Planned start date
- Planned end date
- Actual start date
- Actual end date
- Task status
- Percentage complete
- Delay days
- Reason for delay

Possible analysis:

- Planned versus actual completion
- Delayed tasks
- Average project delay
- Common causes of delay
- Relationship between delays and costs

Quality and defects:

- Inspection ID
- Project ID
- Inspection date
- Area or task inspected
- Defect type
- Severity
- Corrective action
- Rework cost
- Resolution date
- Inspection result

Possible analysis:

- Defect by project
- Rework costs
- Common quality issues
- Relationship between defects and project delay

1/3 Received entire data in XLSX format.

The screenshot displays the WPS Office interface with an Excel spreadsheet. The spreadsheet contains a table of invoice data, organized into sections for different companies. Each section includes a header row with company name, date, and a summary row with totals. The data rows list various items with their respective quantities and prices.

Company	Date	Item	Quantity	Price	Total
Hitoba Bvba	18/10/2023	Levensmiddelen	0.00	152.20	HT-202302578
		Tenue te vanderen BTW	28.41	0.00	HT-202302578
		Aankopen grondstoffen	125.79	0.00	HT-202302578
		BTW-vakken	50 = 26.41	81 = 125.79	
Hitoba Bvba	16/10/2023	Levensmiddelen	0.00	48.12	HT-202302555
		Tenue te vanderen BTW	8.35	0.00	HT-202302555
		Aankopen grondstoffen	38.77	0.00	HT-202302555
		BTW-vakken	50 = 8.35	81 = 38.77	
Oces Group Sprl	17/10/2023	Levensmiddelen	0.00	341.08	OCEB-202319409
		Tenue te vanderen BTW	59.20	0.00	OCEB-202319409
		Aankopen grondstoffen	281.88	0.00	OCEB-202319409
		BTW-vakken	50 = 59.20	81 = 281.88	
Hitoba Bvba	18/10/2023	Levensmiddelen	0.00	74.95	HT-202301813
		Tenue te vanderen BTW	13.01	0.00	HT-202301813
		Aankopen grondstoffen	61.94	0.00	HT-202301813
		BTW-vakken	50 = 13.01	81 = 61.94	
Oces Group Sprl	01/10/2023	Levensmiddelen	0.00	117.89	OCEB-202315471
		Tenue te vanderen BTW	20.43	0.00	OCEB-202315471
		Aankopen grondstoffen	97.26	0.00	OCEB-202315471
		BTW-vakken	50 = 20.43	81 = 97.26	
Hitoba Bvba	10/10/2023	Levensmiddelen	0.00	178.23	HT-202302463
		Tenue te vanderen BTW	30.59	0.00	HT-202302463
		Aankopen grondstoffen	145.64	0.00	HT-202302463
		BTW-vakken	50 = 30.59	81 = 145.64	
Hitoba Bvba	12/10/2023	Levensmiddelen	0.00	105.50	HT-202302508
		Tenue te vanderen BTW	18.31	0.00	HT-202302508
		Aankopen grondstoffen	67.19	0.00	HT-202302508
		BTW-vakken	50 = 18.31	81 = 67.19	
Nokta Bouwmaterialen B.V.B.A.	10/10/2023	Levensmiddelen	0.00	688.87	VP060125
		Tenue te vanderen BTW	119.73	0.00	VP060125
		Aankopen grondstoffen	570.14	0.00	VP060125
		BTW-vakken	50 = 119.73	81 = 570.14	

2/3 Ran a Python script to extract the required information from the invoices and organize it into a structured CSV file for further analysis in tools such as SSMS and Power BI.

```
import re
import os
import pandas as pd

file = pd.read_excel(

r'C:\python\bouwfirma\bouwfirma_data\data\raw\bouwfirma\2023 ads-aanverdagboek.xlsx',
    header=None
)
```

```
# Excel range E14:CY2676
data = file.iloc[13:2676, 4:104].copy()
data = data.fillna('').astype(str)
```

```
# Looser patterns
invoice_start_pattern =
re.compile(r'^\s*([FC])\s*AK\s*(\d+)\s*H\s*(\d+(?:\.\d)?)\b',
re.IGNORECASE)
account_pattern = re.compile(r'^\s*(\d{6})\s+(.*\S)\s*$')
date_pattern = re.compile(r'(\d{4}-\d{2}-\d{2})')
```

```
records = []
current_record = None
```

```
def to_number(value):
    try:
```

```
    return float(str(value).replace(",", "."))
except ValueError:
    return None
```

```
for _, row in data.iterrows():
    row_text = " ".join(row.fillna("").astype(str).tolist())
    row_text = re.sub(r"\s+", " ", row_text).strip()
```

```
if not row_text:
    continue
```

```
# Detect invoice header
```

```
m = invoice_start_pattern.match(row_text)
```

```
if m:
```

```
    prefix = m.group(1).upper()
```

```
    ak_num = m.group(2)
```

```
    h_num = f"{int(float(m.group(3))):02d}"
```

```
# Find the first two dates anywhere in the row
```

```
dates = date_pattern.findall(row_text)
```

```
start_date = dates[0] if len(dates) > 0 else None
```

```
end_date = dates[1] if len(dates) > 1 else None
```

```
# Remove the prefix "F AK ... H .." from the row
```

```
after_header = row_text[m.end():].strip()
```

Supplier is everything before the first date

supplier = after_header

if start_date:

supplier = after_header.split(start_date)[0].strip()

current_record = {

"Invoice Number": f"{prefix} AK {ak_num} H {h_num}",

"Supplier": supplier,

"Start_Date": start_date,

"End_Date": end_date,

"Full_Price": None,

"BTW-receive": None,

"BTW-to-Pay": None,

"Product_price": None,

"%_of_sum": None,

"Other_expansies": None,

"Private": None

}

records.append(current_record)

continue

Detect account/detail rows under current invoice

if current_record:

match = account_pattern.match(row_text)

if match:

rest = match.group(2).strip()

label = rest.lower()

Extract percentages like 21%

pct = re.search(r'(\d+%)', rest)

Extract decimal numbers only, so we ignore things
like VF060290

decimals = re.findall(r'\d+[.]\d+', rest)

decimals = [to_number(x) for x in decimals if
to_number(x) is not None]

if "leveranciers" in label:

Example: Leveranciers 0 48.67 VF060290 -> want
48.67

if decimals:

current_record["Full_Price"] = decimals[-1]

elif "terug te vorderen btw" in label:

Example: 8.45 0 -> want 8.45

if decimals:

current_record["BTW-receive"] = decimals[0]

elif "btw te betalen" in label:

if decimals:

current_record["BTW-to-Pay"] = decimals[0]

elif "aankopen grondstoffen" in label:

Example: 40.22 0 VF060290 21% -> want 40.22 and 21%

if decimals:

current_record["Product_price"] = decimals[0]

if pct:

current_record["%_of_sum"] = pct.group(1)

elif "restaurantkosten" in label or "onderhoud & herstellingen rollend materieel" in label:

if decimals:

current_record["Other_expansies"] = decimals[0]

elif "relatiegeschenken" in label or "r/c bedrijfsleider" in label:

if decimals:

current_record["Private"] = decimals[0]

df = pd.DataFrame(records)

output_path =

r'C:\python\bouwfirma\bouwfirma_data\output\bouwfirma_structured_output.csv'

df.to_csv(output_path, index=False, encoding='utf-8-sig')

print(f"Saved to: {output_path}")

print(f"Rows written: {len(df)}")

print(df.head(10).to_string(index=False))

print(f"Current working directory: {os.getcwd()}")

3/3 After running the code on the source file, I noticed that many cells contained empty or NULL values.

To investigate these gaps efficiently, I opened SQL Server Management Studio (SSMS) alongside my Python environment. I used the Python script to identify the exact invoices containing missing information, then reviewed those invoices and manually entered the missing values into the appropriate cells in the SSMS table. This approach was more efficient than spending additional time modifying the extraction script to capture every piece of information from the inconsistently formatted invoices.

	Supplier	Start_Date	End_Date	Full_Price	BTW-receive	BTW-to-Pay	Product_price	% of sum	Other_expansies	Private
1	Supplier									
2	Nokta Bouwmaterialen B.V.B.A.	16/10/2023	15/11/2023	48.67	8.45		40.22	21%		
3	Hitobo Bvba	19/10/2023	19/10/2023	152.2	26.41		125.79	21%		
4	Hitobo Bvba	16/10/2023	16/10/2023	48.12	8.35		39.77	21%		
5	Oces Group Sprl	17/10/2023	17/10/2023	341.08	59.2		281.88	21%		
6	Hitobo Bvba	18/10/2023	18/10/2023	74.95	13.01		61.94	21%		
7	Oces Group Sprl	01/10/2023	01/10/2023	117.69	20.43		97.26	21%		
8	Hitobo Bvba	10/10/2023	10/10/2023	176.23	30.59		145.64	21%		
9	Hitobo Bvba	12/10/2023	12/10/2023	105.5	18.31		87.19	21%		
10	Nokta Bouwmaterialen B.V.B.A.	10/10/2023	10/10/2023	689.87	119.73		570.14	21%		
11	Probumat Bvba	02/10/2023	02/10/2023	75.43	15.84	15.84	30.3	21%		
12	LUKOIL Belgium n.v.	15/09/2023	15/10/2023	352	61.09		290.91	21%		
13	LUKOIL Belgium n.v.	15/07/2023	14/08/2023	385.02	66.82			21%		
14	LUKOIL Belgium n.v.	15/08/2023	14/09/2023	310.14	53.83			21%		
15	LUKOIL Belgium n.v.	30/06/2023	30/07/2023	242.32	42.06			21%		
16	LUKOIL Belgium n.v.	31/07/2023	31/07/2023	241.67	41.94			21%		
17	LUKOIL Belgium n.v.	15/06/2023	15/06/2023	476.51	82.7			21%		
18	LUKOIL Belgium n.v.	15/05/2023	15/05/2023	443.98	77.05			21%		
19	LUKOIL Belgium n.v.	31/08/2023	31/08/2023	436.18	75.7			21%		
20	Probumat Bvba	17/05/2023	17/05/2023	108.86	18.89		89.97	21%		
21	LUKOIL Belgium n.v.	30/09/2023	30/09/2023	434.67	75.44			21%		
22	IKEA Belgium NV	29/04/2023	29/05/2023	420.98	73.06		347.92	21%		
23	Nokta Bouwmaterialen B.V.B.A.	17/07/2023	17/07/2023	26.3	4.56		21.74	21%		
24	Nokta Bouwmaterialen B.V.B.A.	07/07/2023	07/07/2023	148.37	25.75		122.62	21%		
25	Nokta Bouwmaterialen B.V.B.A.	21/06/2023	21/06/2023	332.69	57.74		274.95	21%		
26	Nokta Bouwmaterialen B.V.B.A.	06/07/2023	06/07/2023	87.82	15.24		72.58	21%		
27	Nokta Bouwmaterialen B.V.B.A.	23/06/2023	23/06/2023	36.3	6.3			21%		
28	Nokta Bouwmaterialen B.V.B.A.	23/06/2023	23/06/2023	223.85	38.85			21%		
29	Nokta Bouwmaterialen B.V.B.A.	30/06/2023	30/06/2023	205.19	35.61		169.58	21%		
30	Nokta Bouwmaterialen B.V.B.A.	22/06/2023	22/06/2023	166.78	28.55		137.83	21%		
31	Nokta Bouwmaterialen B.V.B.A.	07/07/2023	07/07/2023	442.02	76.71		365.31	21%		
32	Nokta Bouwmaterialen B.V.B.A.	24/06/2023	24/06/2023	151.1	26.22		124.88	21%		
33	Nokta Bouwmaterialen B.V.B.A.	23/06/2023	23/06/2023	878.24	152.42		725.82	21%		
34	Nokta Bouwmaterialen B.V.B.A.	26/04/2023	26/04/2023	1092.47	189.6		902.87	21%		
35	Nokta Bouwmaterialen B.V.B.A.	26/04/2023	26/04/2023	130.25	22.61		107.64	21%		
36	Nokta Bouwmaterialen B.V.B.A.	25/04/2023	25/04/2023	151.79	26.34		125.45	21%		

I identified the rows with NULL values in the Full_Price and Product_Price columns and populated the missing values using T-SQL queries.

The image shows a development environment with two main windows. The left window is a code editor for a Python script named `bouwfirma-file_filling.py`. The script uses `pandas` to read an Excel file, identify rows with missing values in the `Full_Price` and `Product_Price` columns, and then uses a regular expression to extract invoice details (F AK 172 H 04, De Roeck Nv, 2022-11-25) to populate these missing values. The right window is Microsoft SQL Server Enterprise Manager, showing a T-SQL query that selects rows from the `FactSales` table where `Full_Price` and `Product_Price` are NULL, ordered by `Start_Date`. Below the query, the results of the query are displayed as a table.

Python Script (bouwfirma-file_filling.py):

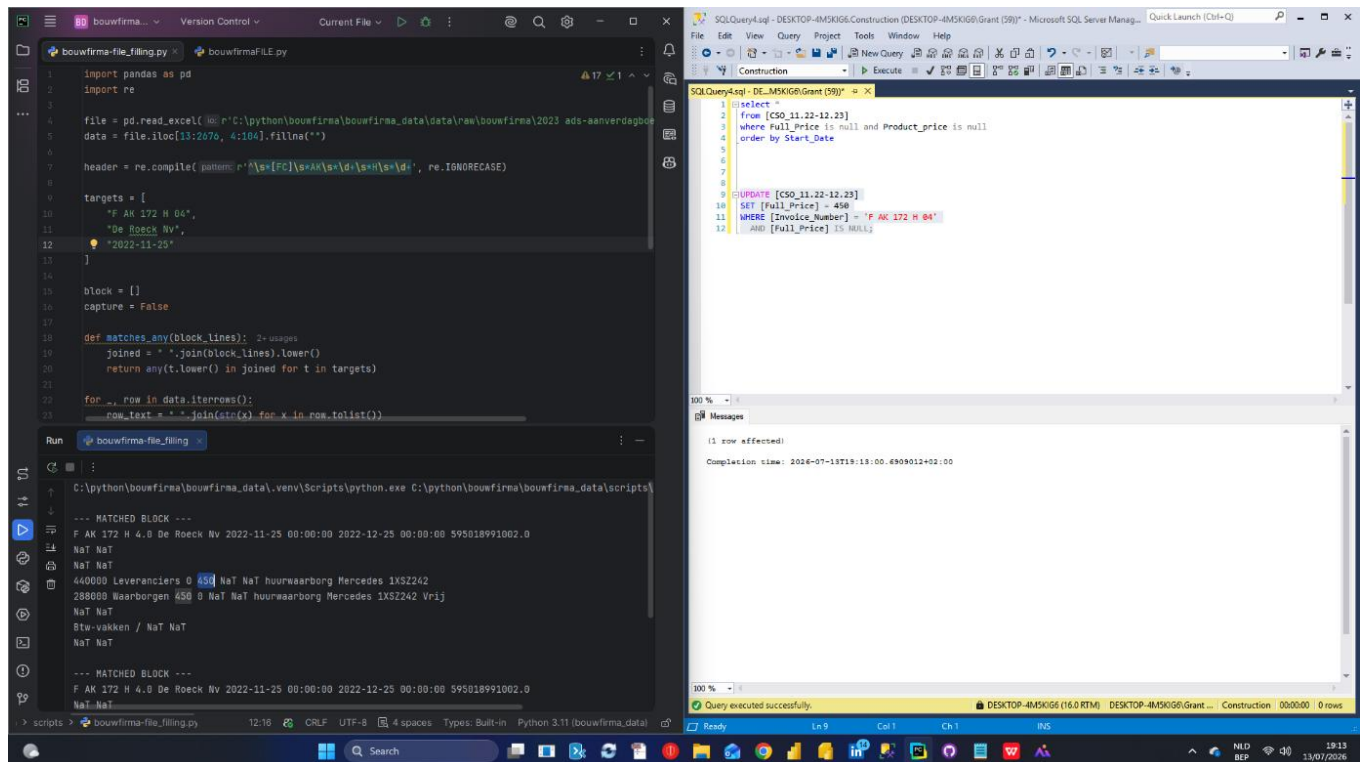
```
1 import pandas as pd
2 import re
3
4 file = pd.read_excel(r'C:\python\bouwfirma\bouwfirma_data\data\raw\bouwfirma\2023-ads-aanverdragbo...
5 data = file.iloc[13:2076, 4:104].fillna("")
6
7 header = re.compile(r'\s*(F|C)\s*(AK|S|D|H|N|S|D)', re.IGNORECASE)
8
9 targets = [
10     "F AK 172 H 04",
11     "De Roeck Nv",
12     "2022-11-25"
13 ]
14
15 block = []
16 capture = False
17
18 def matches_any(block_lines): 2+ usages
19     joined = " ".join(block_lines).lower()
20     return any(t.lower() in joined for t in targets)
21
22 for _, row in data.iterrows():
23     row_text = " ".join(str(x) for x in row.tolist())
```

SQL Query (SQLQuery4.sql - DE...MSK066\Grant (59)):

```
1 select
2     from [C:\SQL\11-22-12-23]
3     where Full_Price is null and Product_price is null
4     order by Start_Date
```

Query Results:

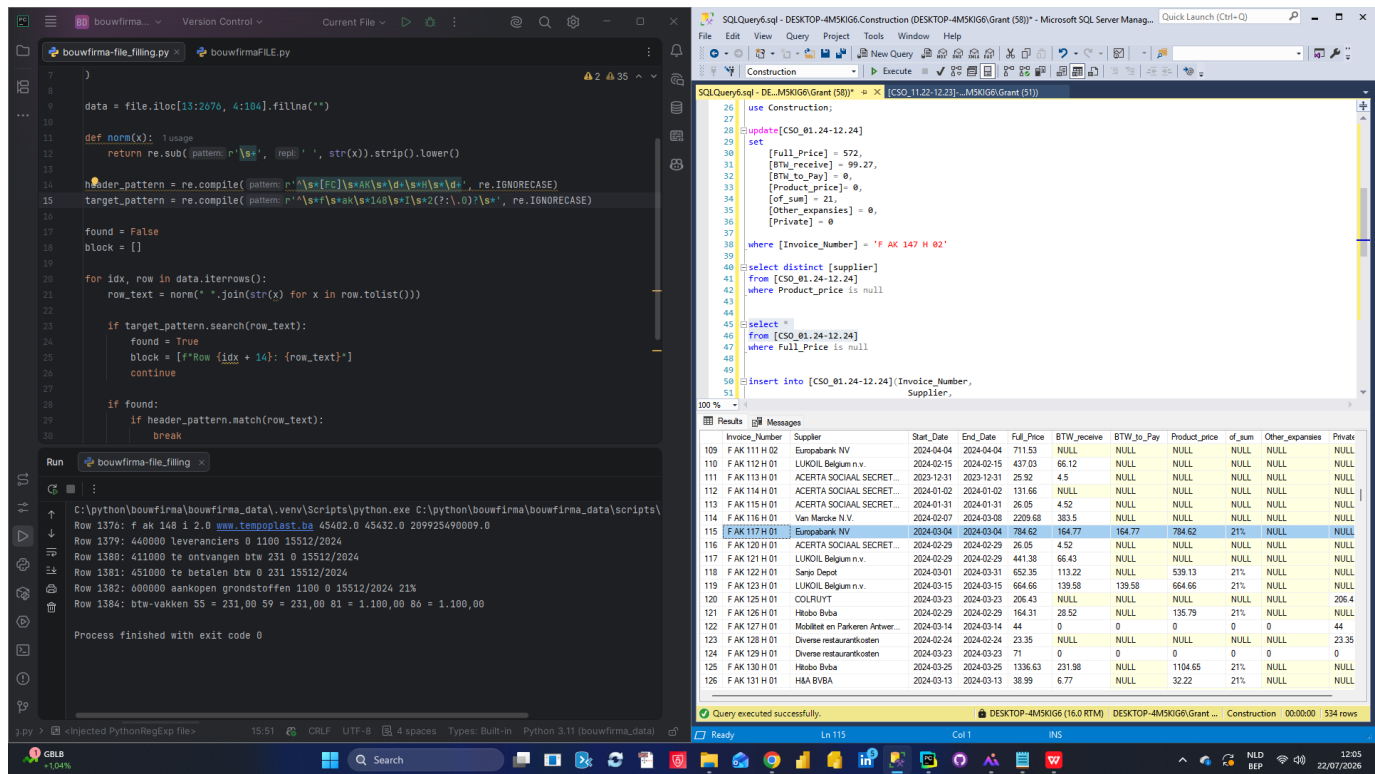
Invoice_Number	Supplier	Start_Date	End_Date	Full_Price	BTW_receive	BTW_to_Pay	Product_price	of_sum	Other_ex
1	F AK 172 H 04	2022-11-25	2022-12-25	NULL	NULL	NULL	NULL	NULL	NULL
2	F AK 181 H 04	2023-02-06	2023-02-06	NULL	NULL	NULL	NULL	NULL	NULL
3	F AK 182 H 04	2023-02-13	2023-02-13	NULL	NULL	NULL	NULL	NULL	NULL
4	F AK 185 H 04	2023-02-14	2023-03-16	NULL	NULL	NULL	NULL	NULL	NULL
5	F AK 189 H 04	2023-03-25	2023-03-25	NULL	NULL	NULL	NULL	NULL	NULL
6	F AK 200 H 05	2023-03-28	2023-05-13	NULL	NULL	NULL	NULL	NULL	NULL
7	F AK 189 H 04	2023-04-20	2023-04-20	NULL	57.0999984741211	NULL	NULL	NULL	NULL
8	F AK 272 H 05	2023-05-06	2023-05-06	NULL	NULL	NULL	NULL	NULL	NULL
9	F AK 273 H 05	2023-06-02	2023-06-02	NULL	NULL	NULL	NULL	0%	NULL
10	F AK 72 H 04	2023-06-07	2023-07-07	NULL	NULL	NULL	NULL	NULL	NULL
11	F AK 73 H 04	2023-06-13	2023-07-13	NULL	NULL	NULL	NULL	NULL	NULL
12	F AK 88 H 04	2023-06-17	2023-07-17	NULL	NULL	NULL	NULL	NULL	NULL
13	F AK 201 H 04	2023-06-25	2023-06-18	NULL	NULL	NULL	NULL	NULL	NULL
14	F AK 92 H 04	2023-07-03	2023-08-02	NULL	NULL	NULL	NULL	NULL	NULL
15	F AK 139 H 04	2023-08-22	2023-08-22	NULL	NULL	NULL	NULL	NULL	NULL
16	F AK 142 H 04	2023-08-22	2023-08-22	NULL	NULL	NULL	NULL	NULL	NULL
17	F AK 170 H 04	2023-09-04	2023-10-04	NULL	NULL	NULL	NULL	NULL	NULL
18	F AK 94 H 04	2023-09-06	2023-09-06	NULL	NULL	NULL	NULL	NULL	NULL



However, I then realized it would be safer and faster to fill in the entire row with the correct information, rather than only

updating the Full_Price column (VAT-inclusive) and using formulas to derive the Product_Price column (VAT-exclusive), which would then be used together with those two columns to calculate the %_of_sum column.

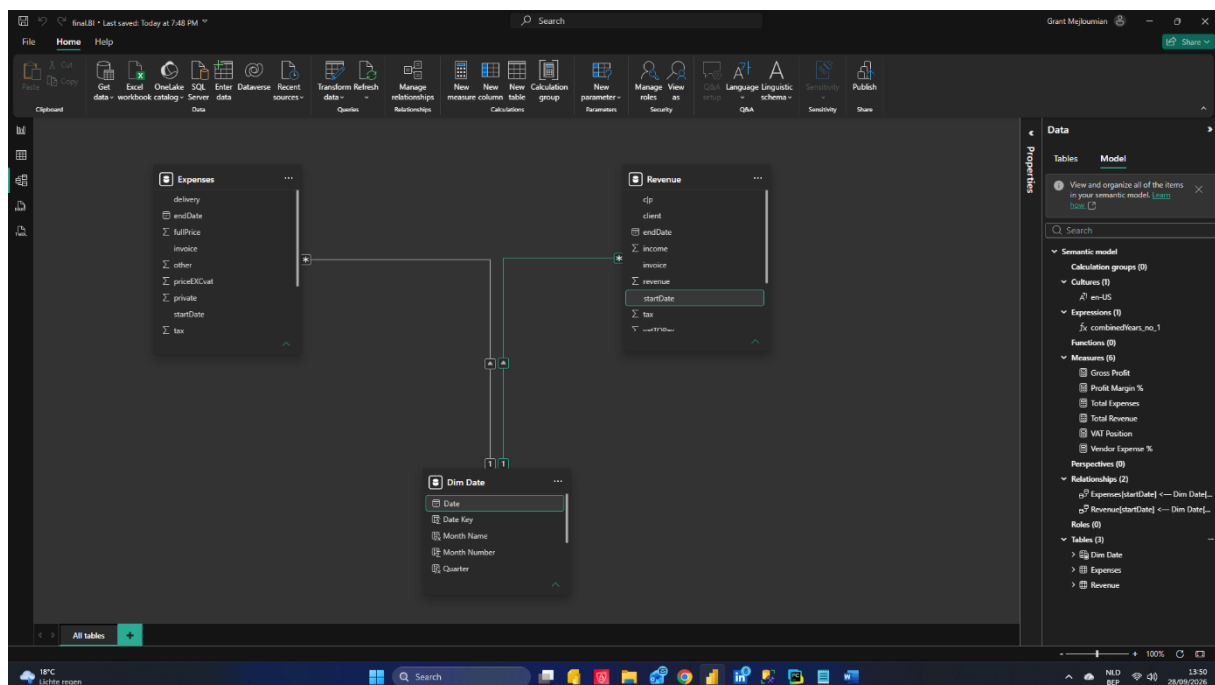
The screenshot displays a development environment with two main windows. The left window, titled 'bouwfirma-file_filing.py', contains a Python script that reads an Excel file, processes data, and searches for specific patterns. The right window, titled 'SQLQuery4.sql - DESKTOP-4MSKIG6\Grant (59)', shows a SQL query and its results. The query is a SELECT statement that filters data from a table named 'C50_11-22-12.23' based on 'Full_Price' and 'Product_price'. The results table shows columns: Invoice_Number, Supplier, Start_Date, End_Date, Full_Price, BTW_receive, BTW_to_Pay, Product_price, of_sum, Other_expenses, Private. The results show data for various invoices, including one for 'f ak 269 h 5.0'.



Using the same approach, I cleaned, completed, and consolidated the purchase files from **2022 through 2025**. I applied the same process to the client payment files, combining the data into a single structured dataset for further analysis in Power BI.

I then moved to Power BI, where I renamed the tables **Expenses** and **Revenue**. I also created an additional **Date** table to establish relationships between the two tables through a shared date dimension.

Date	Year	Month Number	Month Name	Quarter	Year-Month	Year-Month Sort	Date Key
11 November 2022	2022	11	November	Q4	2022-11	202211	20221111
12 November 2022	2022	11	November	Q4	2022-11	202211	20221112
13 November 2022	2022	11	November	Q4	2022-11	202211	20221113
14 November 2022	2022	11	November	Q4	2022-11	202211	20221114
15 November 2022	2022	11	November	Q4	2022-11	202211	20221115
16 November 2022	2022	11	November	Q4	2022-11	202211	20221116
17 November 2022	2022	11	November	Q4	2022-11	202211	20221117
18 November 2022	2022	11	November	Q4	2022-11	202211	20221118
19 November 2022	2022	11	November	Q4	2022-11	202211	20221119
20 November 2022	2022	11	November	Q4	2022-11	202211	20221120
21 November 2022	2022	11	November	Q4	2022-11	202211	20221121
22 November 2022	2022	11	November	Q4	2022-11	202211	20221122
23 November 2022	2022	11	November	Q4	2022-11	202211	20221123
24 November 2022	2022	11	November	Q4	2022-11	202211	20221124
25 November 2022	2022	11	November	Q4	2022-11	202211	20221125
26 November 2022	2022	11	November	Q4	2022-11	202211	20221126
27 November 2022	2022	11	November	Q4	2022-11	202211	20221127
28 November 2022	2022	11	November	Q4	2022-11	202211	20221128
29 November 2022	2022	11	November	Q4	2022-11	202211	20221129
30 November 2022	2022	11	November	Q4	2022-11	202211	20221130
01 December 2022	2022	12	December	Q4	2022-12	202212	20221201
02 December 2022	2022	12	December	Q4	2022-12	202212	20221202
03 December 2022	2022	12	December	Q4	2022-12	202212	20221203
04 December 2022	2022	12	December	Q4	2022-12	202212	20221204
05 December 2022	2022	12	December	Q4	2022-12	202212	20221205
06 December 2022	2022	12	December	Q4	2022-12	202212	20221206
07 December 2022	2022	12	December	Q4	2022-12	202212	20221207
08 December 2022	2022	12	December	Q4	2022-12	202212	20221208
09 December 2022	2022	12	December	Q4	2022-12	202212	20221209
10 December 2022	2022	12	December	Q4	2022-12	202212	20221210
11 December 2022	2022	12	December	Q4	2022-12	202212	20221211
12 December 2022	2022	12	December	Q4	2022-12	202212	20221212
13 December 2022	2022	12	December	Q4	2022-12	202212	20221213
14 December 2022	2022	12	December	Q4	2022-12	202212	20221214
15 December 2022	2022	12	December	Q4	2022-12	202212	20221215
16 December 2022	2022	12	December	Q4	2022-12	202212	20221216
17 December 2022	2022	12	December	Q4	2022-12	202212	20221217

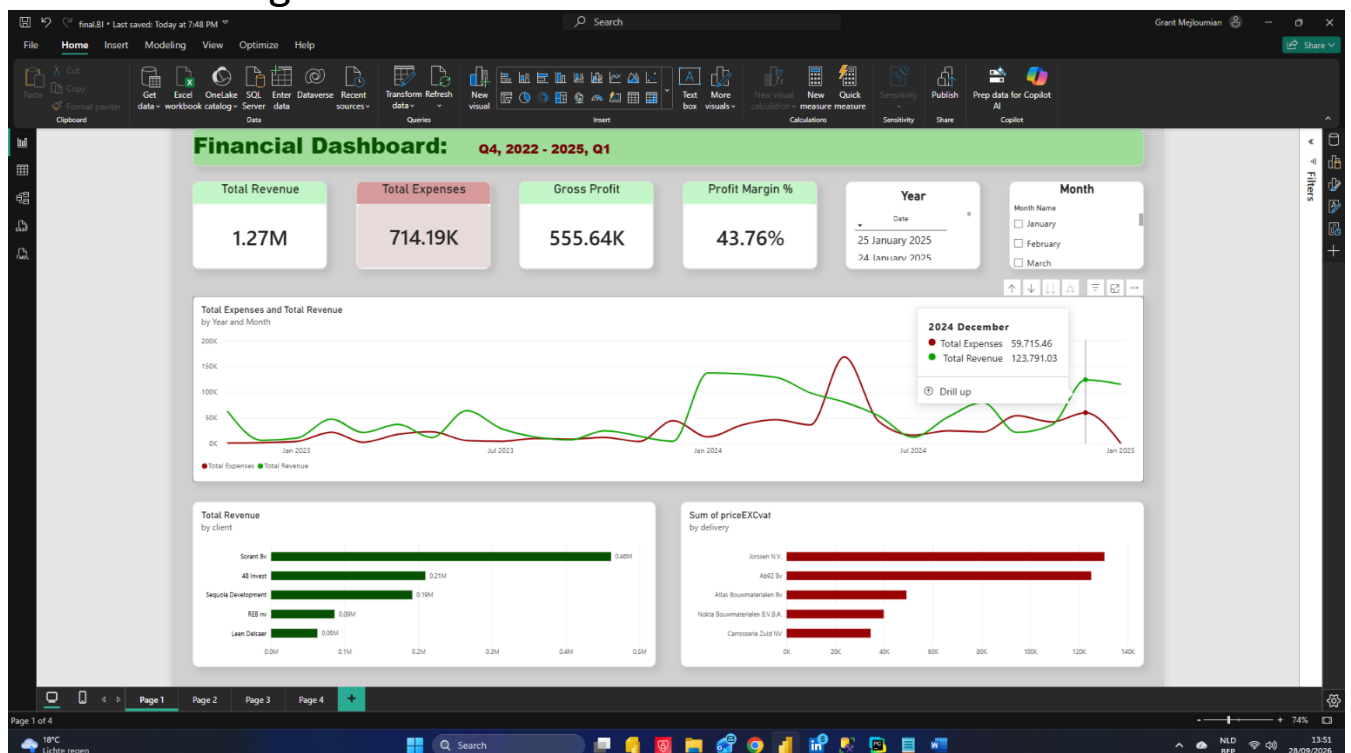


After creating the additional table and establishing the necessary relationships, I used a minimalist visual approach to address the key business questions on the dashboard.

Dashboard Overview

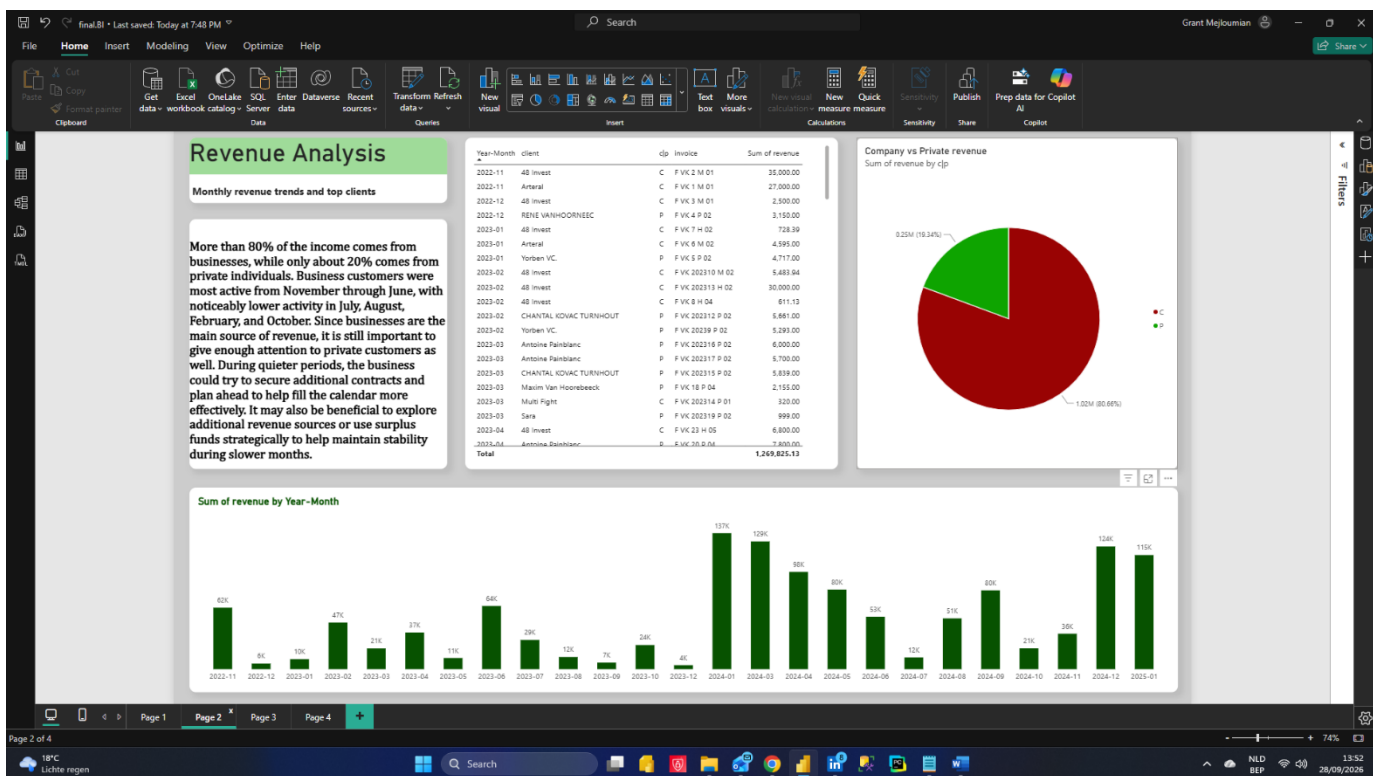
This dashboard provides a high-level view of the company's financial performance by comparing **Revenue** and **Expenses** over time. It is built on the available transaction dataset (purchase records and received payments) and uses a shared **Date** table for time-based analysis. The visuals highlight spending and income trends, making it easier to identify profitable periods and potential cost pressure points. The goal is to support clear, data-driven decisions with a simple and readable layout.

- **Revenue vs. Expenses trend:** shows financial performance over time.
- **Category/segment breakdown:** identifies where the highest costs or income are concentrated.
- **Period filters:** allow quick comparison across different time ranges.



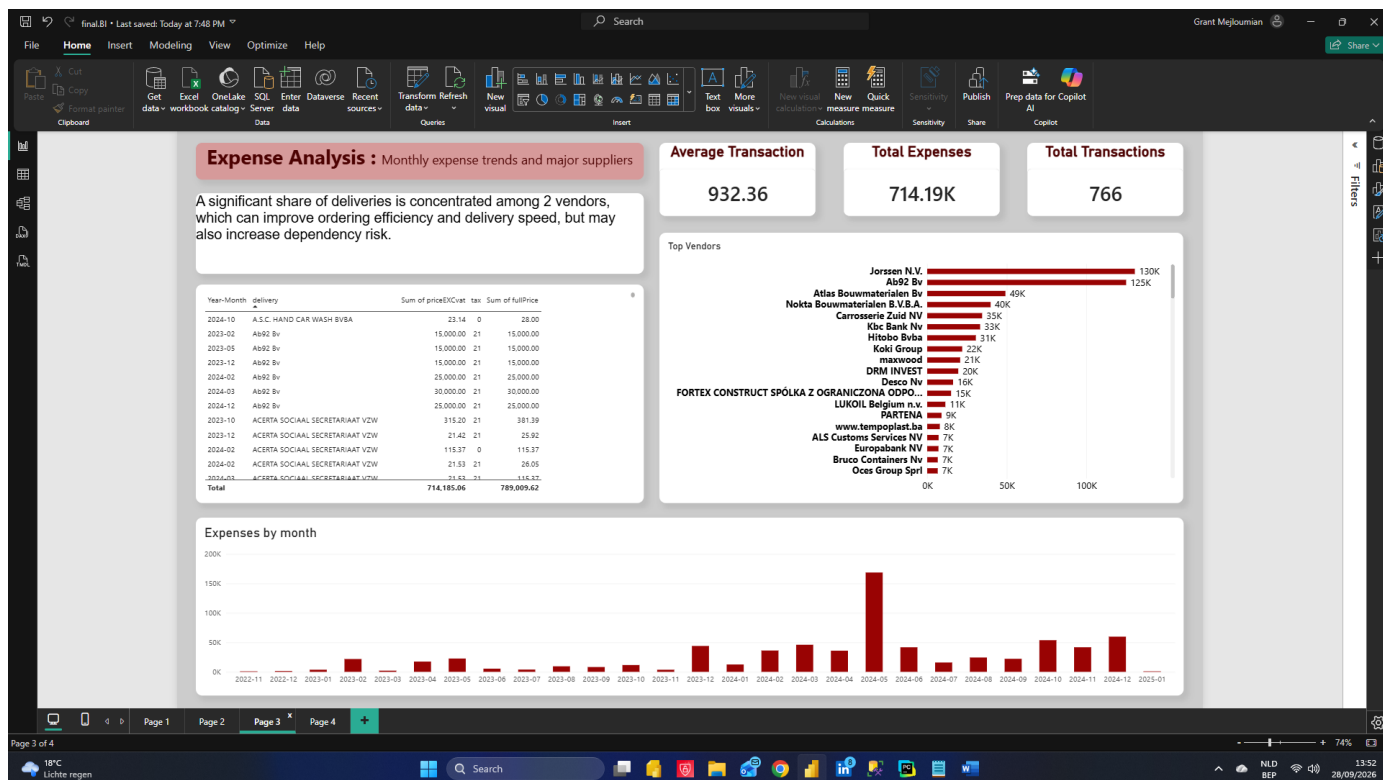
Revenue Page Overview

This page focuses on revenue performance, highlighting trends over time and key contributing segments. The text box in the dashboard summarizes the main findings and business interpretation.



Expenses Page Overview

This page focuses on the company's expense performance, highlighting spending trends over time and the main cost categories. The text box within the dashboard summarizes the key findings and their potential impact on the company's financial performance.



Profitability Page Overview

This page presents the company's **net profit and profit margin percentage**. The visuals show how profitability changes over time, helping identify periods of stronger or weaker financial performance. The text box within the dashboard summarizes the key results and their potential business implications.

